Audit Secure Horizons — rapport & propositions de réponse

Projet : secure-horizons (Équipe 20 — Guardians of Value) · Audit en lecture seule, puis propositions de correction · 2026-10-11
Chaque problème est suivi d'une **proposition de réponse recommandée** (la plus pertinente selon l'auditeur), à valider avant implémentation.

Avant-propos : les mots-clés

Terme	Explication simple
On-chain / off-chain	On-chain = exécuté par la blockchain, infalsifiable. Off-chain = sur un serveur normal à côté, plus souple mais non garanti.
Smart contract	Programme déposé sur la blockchain, au code figé, qui s'exécute selon ses règles.
Mint / burn	Mint = créer des tokens à partir de rien. Burn = les détruire.
Clé de signature	Le secret qui autorise une opération. Problème central : si elle est *volée*, le système prend l'attaquant pour un utilisateur légitime.
Évasion / contournement	Passer *sans* déclencher l'alarme (se faufiler), pas « casser » la défense.
Faux positif / faux négatif	Faux positif = alarme à tort (gênant). Faux négatif = pas d'alarme alors qu'il y a attaque (dangereux).
Token Ghost	Jeton temporaire émis à chaque transfert ; converti 1:1 en vrai token après une fenêtre sans anomalie, détruit si fraude confirmée.

Résumé

Le **cœur on-chain est solide** (contrat `ProtectedMinter` : montant/destinataire dérivés, taux figé, un seul mint immuable, pause 3/3). La faiblesse est **la couche de détection** et quelques points de robustesse dans la réponse. Neuf problèmes sont documentés ci-dessous, chacun avec une proposition de correction. Les trois

priorités : (1) rendre la détection non contournable par mise en forme lente du trafic ; (2) remplacer la logique « ET » du discriminant ; (3) implémenter le lignage d'origine du Ghost — sans lui, la promesse de récupération/saisie ne tient pas.

Problèmes détaillés et propositions de réponse

4.1 — Empoisonnement de la baseline CRITIQUE

Fichier : `detection/detector.py` (`_close_open_slot`).

Mots : *baseline* = la « normale » calculée sur le passé récent. *z-score* = « à combien d'écarts-types au-dessus de la moyenne » (à quel point c'est inhabituel). Seuil ici z=3.

Le problème : le détecteur ajoute à sa « normale » *tout* moment qui n'a pas déclenché l'alerte. Un attaquant patient envoie des volumes *juste sous* le seuil ; chacun est jugé normal et fait monter la baseline. Après ~7 répétitions, la vraie grosse attaque ne dépasse plus le seuil → faux négatif.

> **Proposition recommandée**
>
> **Zone morte + quarantaine de la baseline.** Ne plus apprendre sur les slots « approchants » : exclure de la baseline tout slot dont le z-score dépasse `seuil - marge` (ex. marge = 1), pas seulement les slots qui déclenchent. En complément, borner la vitesse de dérive admissible de la médiane entre deux fenêtres (si la « normale » grimpe trop vite, on gèle la mise à jour et on lève un signal). Coût faible, change seulement la règle d'inclusion dans `_close_open_slot`, testable en rejouant un trafic « montée lente ».

4.2 — Discriminant en « ET » : battre une seule condition suffit CRITIQUE

Fichier : `detection/detector.py` — `volume_z >= seuil AND hhi_z >= seuil`.

Mots : *dispersion/concentration* = l'argent part vers beaucoup (dispersé) ou peu (concentré) d'adresses. *HHI* = indice de concentration (élevé = concentré).

Le problème : l'alerte exige volume élevé *ET* concentration forte. L'attaquant n'a donc qu'à en éviter une : éparpiller le volume sur beaucoup d'adresses garde le HHI bas → pas d'alerte, même sur gros volume.

> **Proposition recommandée**
>
> **Score de risque continu au lieu d'un ET binaire.** Remplacer le double seuil par un score combiné (ex. `score = w1·volume_z + w2·hhi_z + w3·fanout_z`) qui déclenche au-dessus d'un seuil unique — ainsi un volume très anormal *seul* peut déjà alerter. Ajouter un troisième signal **anti-fan-out** : nombre d'adresses destinataires neuves par slot (un éparpillement soudain vers des adresses jamais vues est lui-même suspect). On garde l'esprit « volume + structure » mais sans le trou du « ET ». À calibrer sur le trafic synthétique (règle « aucun chiffre sans mesure »).

4.3 — Récupération 0 % une fois l'argent propagé (contamination prospective) ÉLEVÉ

Fichier : `data/ghost_ledger.py`, mesuré par `data/late_marking.py`.

Mots : *prospectif* = on ne marque le voleur qu'au moment de la détection ; ce qu'il a envoyé avant portait une suspicion de 0. *Lignage d'origine* = on étiquette la source frauduleuse ; toute sa descendance reste suspecte où qu'elle aille.

Le problème : votre propre mesure donne 0 % récupéré en prospectif dès que l'argent a circulé, contre 100 % avec le lignage (si l'alerte précède l'échéance). Le modèle implémenté est le prospectif.

> **Proposition recommandée**
>
> **Implémenter le lignage d'origine par tranche (piste a).** Chaque tranche de Ghost porte l'identifiant de son origine (le mint/émission d'où elle provient). Une alerte vise une *origine*, et toute tranche non convertie issue de cette origine devient saisissable, peu importe chez qui elle est. C'est exactement ce que la nouvelle mécanique séquestre/saisie (`docs/GHOST_MARKET_SEIZURE.md`, déjà mergée) requiert pour fonctionner. Prototypable d'abord dans le registre hors-chaine, chiffré par un sweep « délai de détection vs part saisie », avant de le porter au contrat.

4.4 — Pas d'idempotence : rejeu d'alerte → déni de service ÉLEVÉ

Fichier : `response/orchestrator.py` (`handle_alert`, `apply_release`).

Mots : *idempotence* = répéter l'action ne produit pas d'effet en plus. *Replay* = renvoyer un message légitime pour en abuser. *DoS* = bloquer le service pour les honnêtes gens.

Le problème : aucune trace des alertes/décisions déjà traitées. Rejouer la même alerte relance la réponse ; combiné au reset du plafond de retard, un seul payload capturé peut geler un périmètre en permanence (DoS sur la conversion).

> **Proposition recommandée**
>
> **Registre de nonces + ancrage du plafond au premier retard.** Tenir un ensemble des `alert_id` et `decision_id` déjà consommés ; toute répétition est ignorée (journalisée comme doublon, sans effet). Et ancrer le compteur « durée max de retard » au *premier* retard du périmètre (ne pas le remettre à zéro à chaque nouveau retard). Correctif court et local, supprime le DoS et le gonflement du journal.

4.5 — Quorum multisig réglable à 1 ÉLEVÉ

Fichier : `response/orchestrator.py` (validation de `ResponsePolicy`).

Mots : *multisig* = plusieurs clés doivent signer. *quorum* = nombre de signatures requises.

Le problème : la vérification est correcte (signataires dédupliqués, tous guardians, `max(quorum, threshold)`), *mais* le code autorise de configurer un quorum de 1. Avec un quorum de 1, une clé volée pourrait détruire/pauser.

> **Proposition recommandée**

> **Interdire un quorum < 2 par construction.** Dans la validation de `ResponsePolicy`, refuser `perimeter_quorum < 2` et `global_pause_quorum < 2` (lever une erreur claire). La protection cesse de dépendre d'un bon réglage : elle devient un invariant. Correctif de quelques lignes + un test.

4.6 — Journal d'audit falsifiable ÉLEVÉ

Fichier : `response/` (sortie JSONL).

Mots : *append-only* = on n'ajoute que des lignes. *hash* = empreinte unique d'un contenu. *chaînage de hash* = chaque ligne inclut l'empreinte de la précédente (maillons soudés).

Le problème : le fichier JSONL peut être édité/tronqué sans détection ; le « append-only DORA » n'est garanti que par le programme en mémoire.

> **Proposition recommandée**

> **Chaînage de hash + numéro de séquence.** Chaque entrée inclut un champ `prev_hash` (empreinte de l'entrée précédente) et un `seq` incrémental ; un vérificateur relit le journal et détecte toute modification/coupure. Argument DORA concret pour le jury, sans dépendance lourde (hachage standard de la bibliothèque Python).

4.7 — Divisions par zéro latentes MOYEN

Fichier : `detection/` (`ratio = suspect / amount`, `seeds_inflow`).

Le problème : aujourd'hui sans danger (les modèles imposent `amount > 0`), mais la protection est ailleurs que là où se fait le calcul — fragile à un futur remaniement.

> **Proposition recommandée**

> **Garde locale explicite.** Ajouter au point de calcul une vérification `if amount == 0: return ratio 0.0` (et idem pour `seeds_inflow`), avec un test dédié. Rend l'invariant local au lieu de dépendre d'un contrat de données distant.

4.8 — Analyse de mint aveugle par défaut MOYEN

Fichier : `detection/` (`check_mint` sans contexte).

Mots : *contexte* = les infos annexes nécessaires à une règle (ex. « quelle demande correspond à ce mint ? »).

Le problème : sans contexte fourni, les règles de mint (dont I3) sont toutes sautées ; l'attaque de détournement 1:1 n'est pas détectée dans le déploiement par défaut.

> **Proposition recommandée**

> **Échouer bruyamment plutôt que silencieusement.** Si aucun fournisseur de contexte n'est branché en déploiement « réel », lever une alerte de configuration *critique* au démarrage (pas un simple warn-once), et refuser de considérer la surveillance de mint comme « active ». Documenter la dépendance au contexte comme pré-requis de déploiement.

4.9 — Destruction « irréversible » réversible dans le modèle FAIBLE

Fichier : `response/` (état DESTROYED).

Le problème : un périmètre « détruit » peut re-rentrer en « ralenti » via une action ultérieure, alors que le burn est décrit comme irréversible — incohérence entre la simulation et la sémantique on-chain visée.

> **Proposition recommandée**

> **État terminal verrouillé.** Faire de DESTROYED un état absorbant : toute transition sortante est refusée (erreur explicite). Aligne le miroir hors-chaine sur le comportement du contrat et évite une démonstration trompeuse.

Plan d'implémentation proposé (après validation)

Ordre	Problème	Effort	Où
1	4.5 Quorum ≥ 2 + 4.4 idempotence	Court	`response/orchestrator.py`
2	4.1 zone morte baseline + 4.2 score continu / anti-fan-out	Moyen	`detection/detector.py`, `detection/params.py`
3	4.3 lignage d'origine (prototype chiffré)	Moyen	`data/ghost_ledger.py`, `data/late_marking.py`
4	4.6 journal hash-chaîné	Court	`response/`
5	4.7 gardes div/0 · 4.8 fail-loud mint · 4.9 état terminal	Court	`detection/`, `response/`

Chaque lot = une branche + une PR + sa suite de tests, mergée après CI verte. Les modules `shared/` et les contrats Solidity ne sont touchés que si vous le validez explicitement (règle de périmètre d'équipe). Les points touchant la conception (score de détection, lignage) seront aussi consignés comme décisions dans `docs/OPEN_QUESTIONS.md`.